

CSC1110/CSC1122: Natural Language Technologies/ Foundations of NLP

Lecture 11a **Large Language Model Wrap-up**

Dr. Ellen Rushe
Assistant Professor
School of Computing
Dublin City University

L2.24 (McNulty Building)
ellen.rushe@dcu.ie

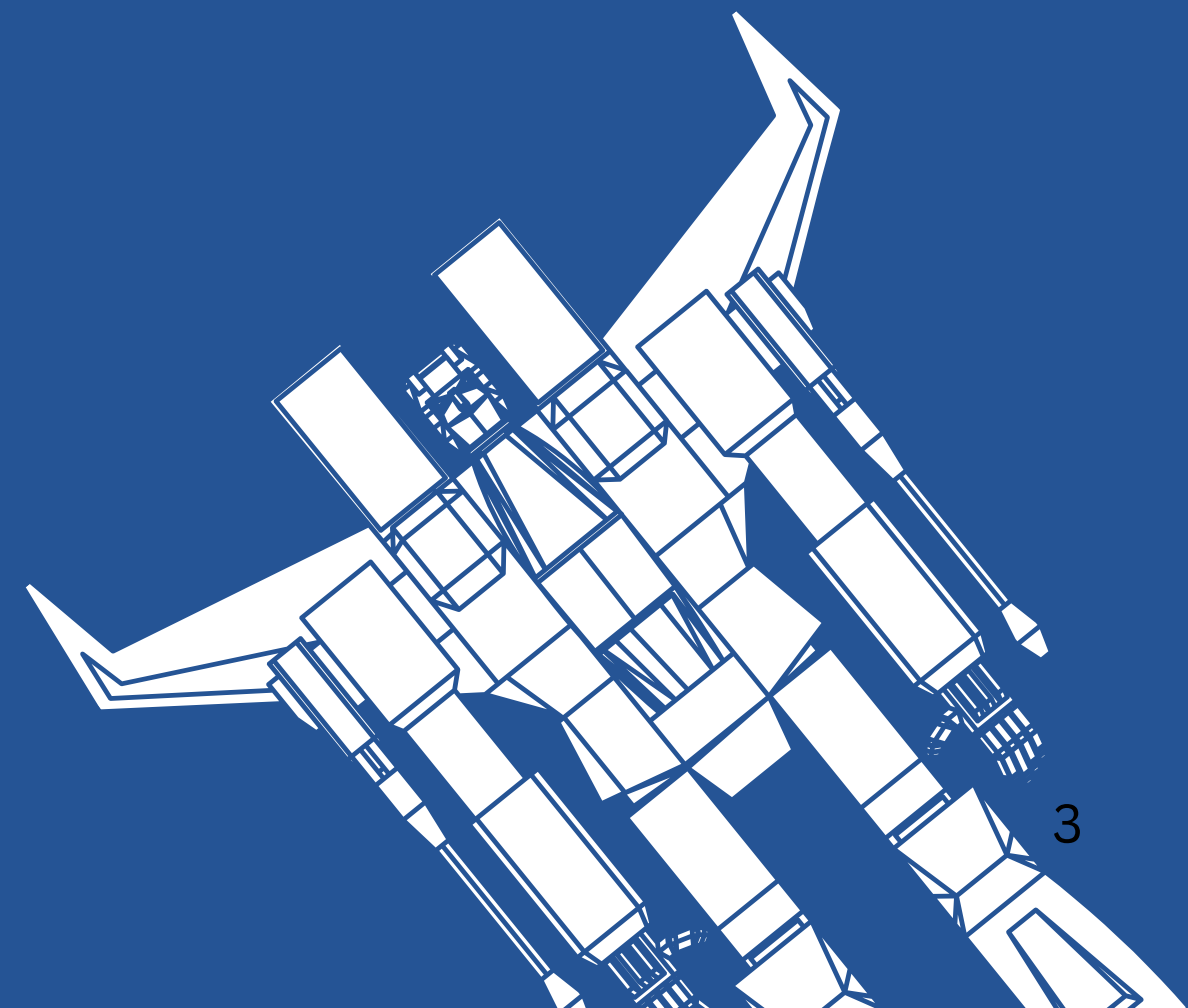
DCU

Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Slides adopted from past course slides by
Maya Popovic on Machine Translation and
slides by Jennifer Foster in past module
iterations.

Where were we?

Transformers



Transformers: The main concepts



Self-attention



Multi-head attention



Positional encoding

Attention:

In General & self-attention

Attention in Neural Networks

General Concept

A network X pays attention to some part of a network Y

In RNNs for machine translation

X = Decoder

Y = Encoder

Decoder pays attention to some part of the Encoder

Attention in Neural Networks

General Concept

A network X pays attention to some part of a network Y



In RNNs for machine translation

X = Decoder

Y = Encoder

Decoder pays attention to some part of the Encoder

However X and Y do not have to be two different networks!

Attention in Neural Networks

General Concept

A network X pays attention to some part of a network Y



In RNNs for machine translation

X = Decoder

Y = Encoder

Decoder pays attention to some part of the Encoder

However X and Y do not have to be two different networks!

If $X=Y$  self-attention

Attention in Neural Networks

General Concept

$X \neq Y$ (Attention in MT RNNs)

One network (decoder) pays attention to some other part of the network (encoder).

$X = Y$ (self-attention)

One network pays attention **to itself** (i.e. its own parts).

Attention in Neural Networks: Inputs

Attention in neural networks takes two types of input



Attention in Neural Networks: Inputs

Attention in neural networks takes two types of input



The idea

→ Find which “keys and values” are most relevant to the current query.

→ in other words: Which “keys and values” should the current “query” pay attention to?

Attention in Neural Networks: Inputs

Keys and values can be the same

→ But they don't **have** to be!

**The terms come from modelling computer memories by
neural networks**

→ Looking for a value stored at a key

→ The key should correspond to the given query

Attention in Neural Networks: Inputs

For attention in MT RNNs



Query (Q)



Decoder's hidden state z_{i-1}

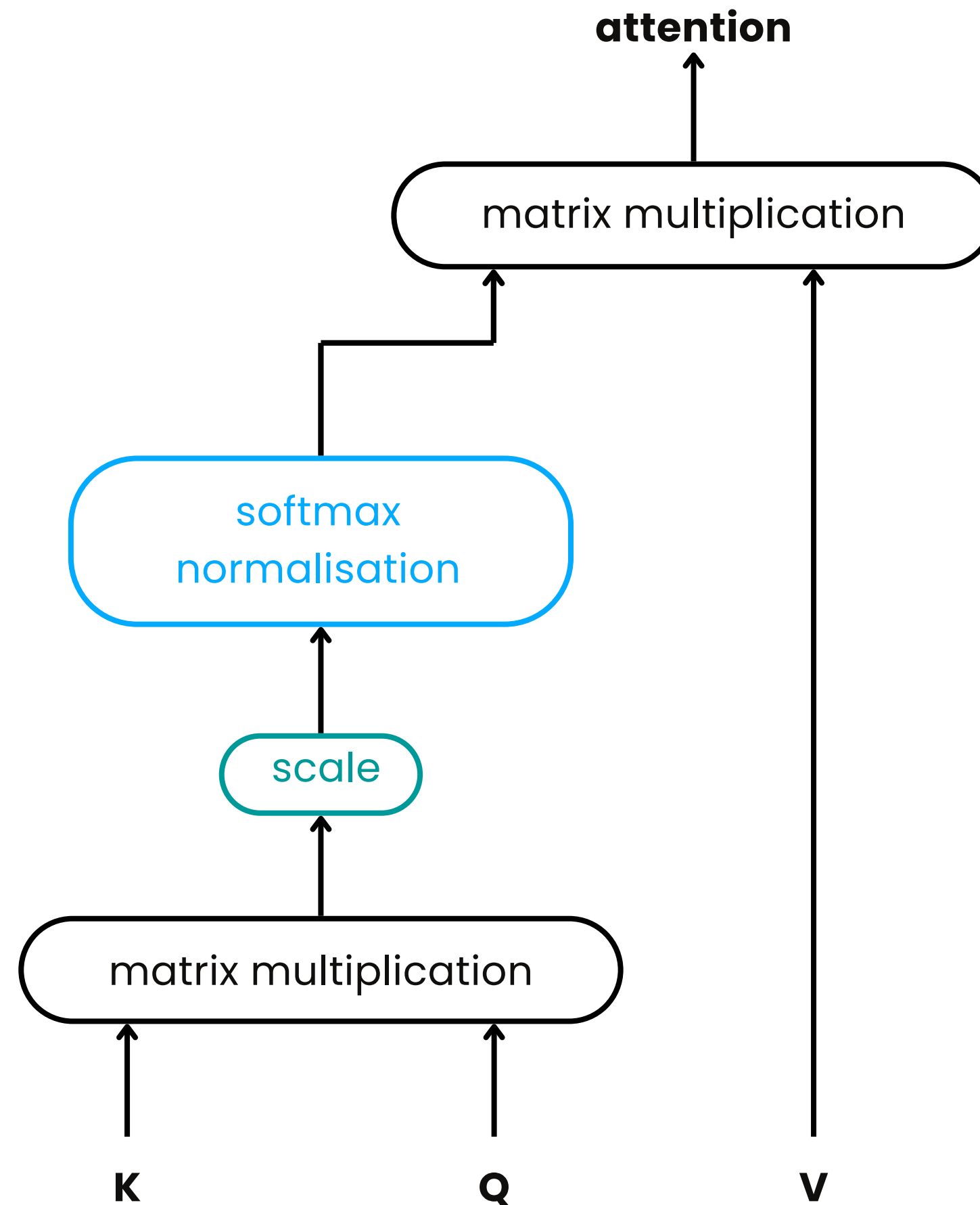


Key/Values (K/V)



Encoders outputs $h_1, \dots, h_j, \dots, h_J$

Dot Product Attention: Graphical Scheme



Attention in MT RNNs: Inputs

Attention in RNNs



Query (Q)

Output of the previous decoder hidden state z_{i-1}



Key/Values (K/V)

All source word representations $h_1, \dots, h_j, \dots, h_J$

Self-attention: inputs

Self-Attention



Query (Q)

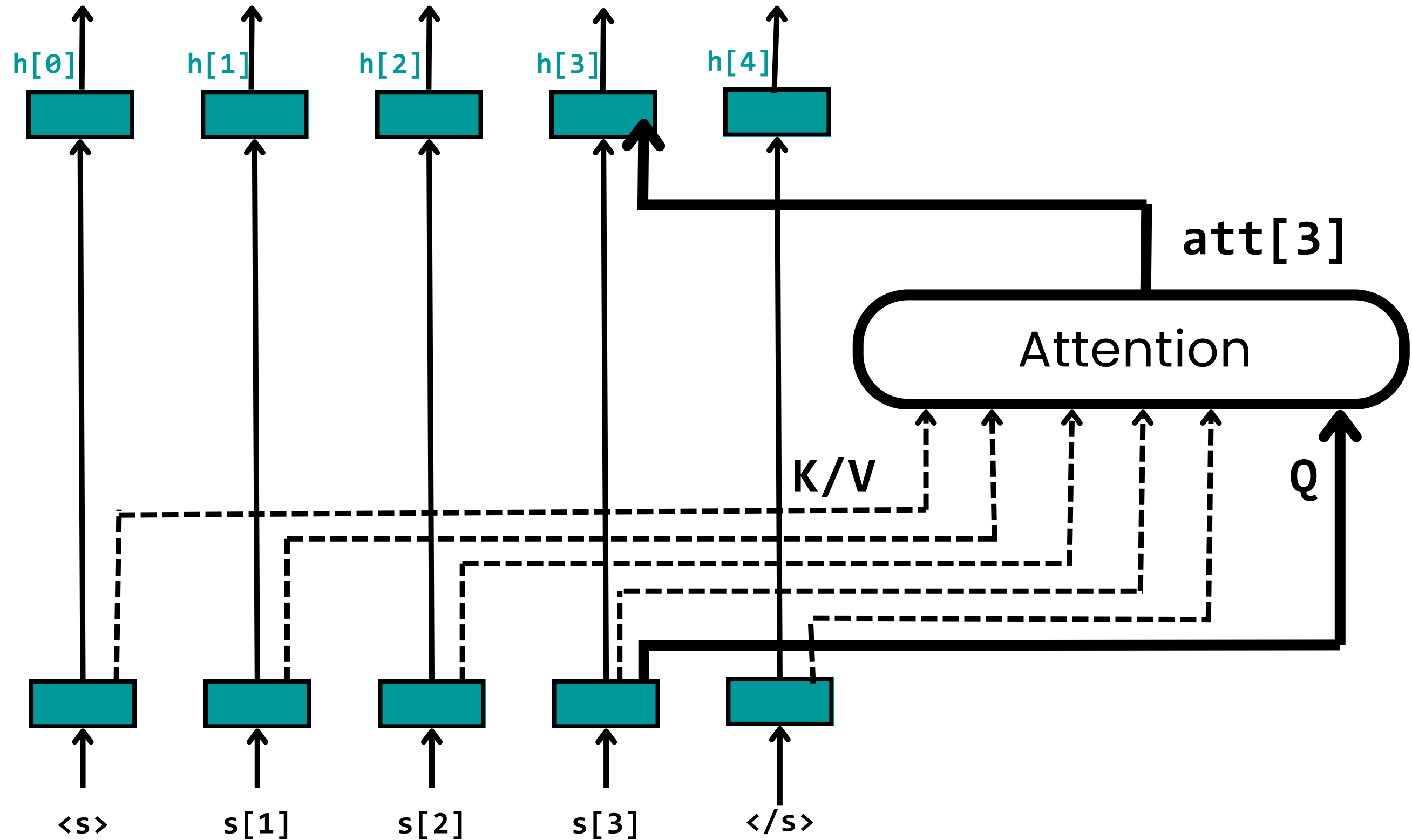
Current output of the network's lower layer.



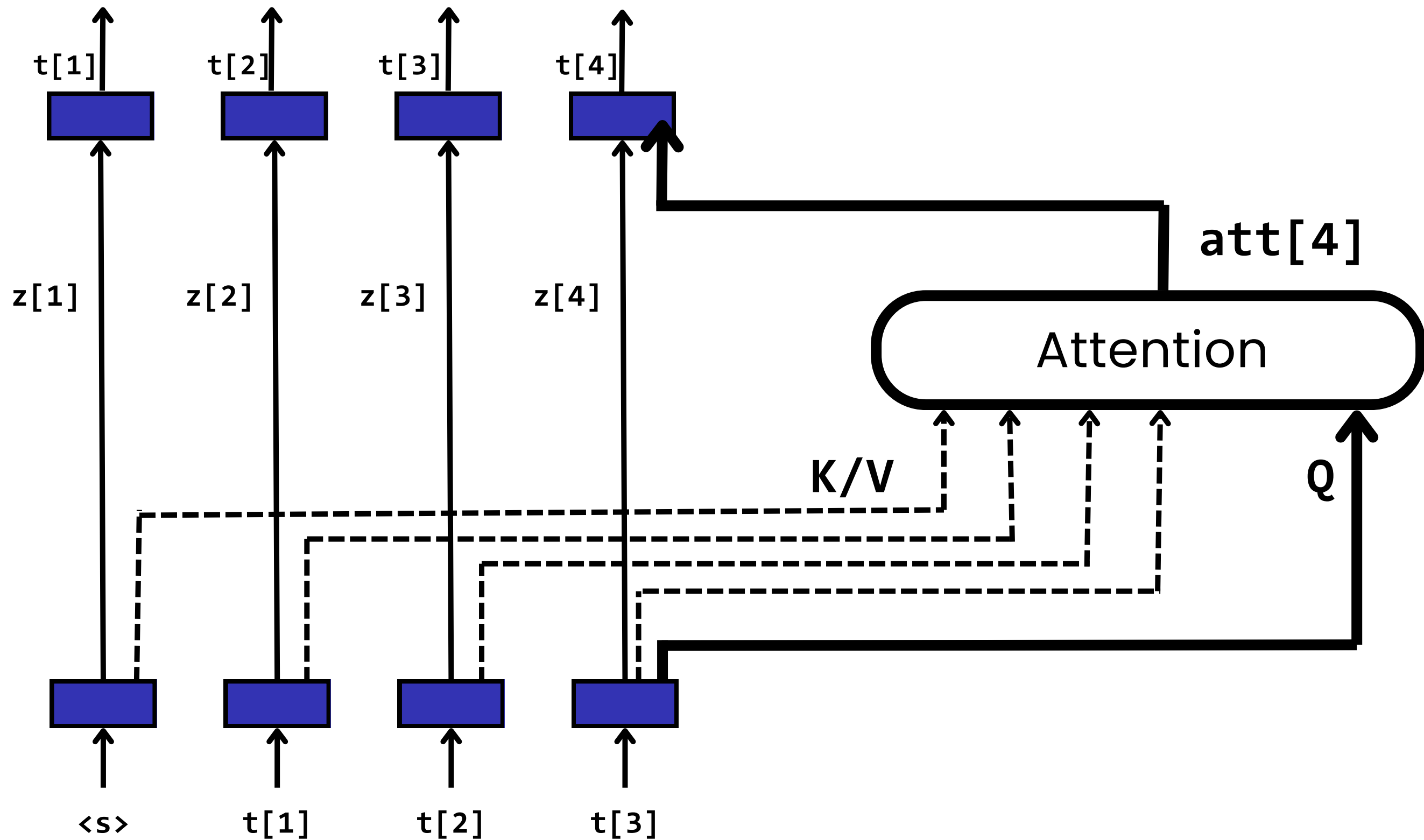
Key/Values (K/V)

All outputs of the network's lower layer

Encoder with self-attention: word s_3



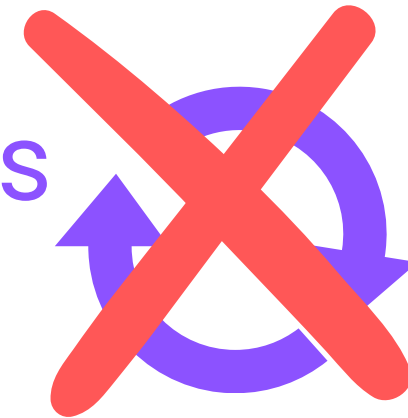
Decoder with self-attention: word t_4



The main and the most important difference

In the encoder/decoder with self-attention

→ There are no recurrent connections



This is the key idea of the transformer architecture.

Multi-head Attention

Several attention blocks running in parallel

↪ M different queries Q_m (instead of one)

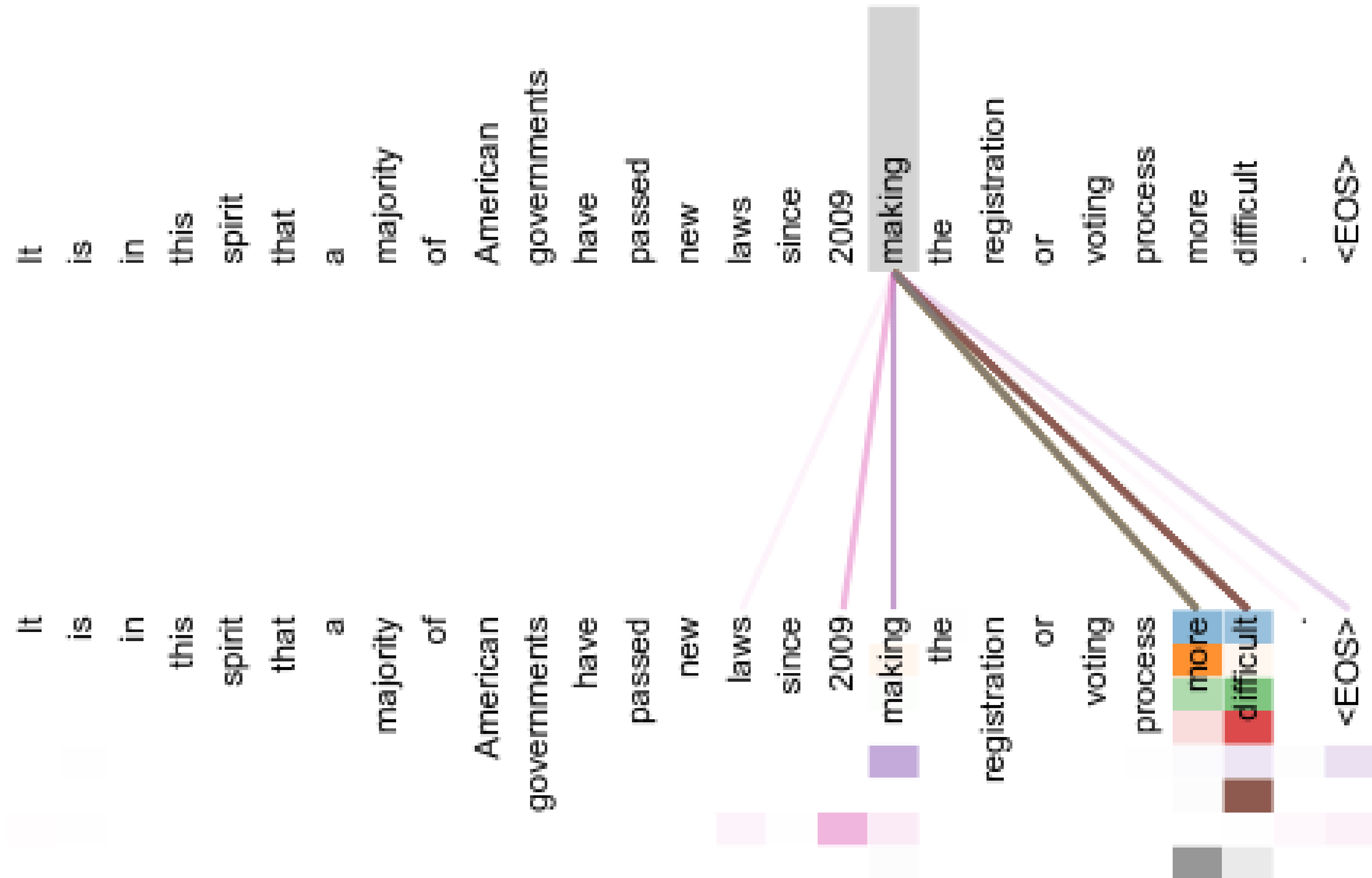
↪ M different outputs att_m combined into one multi-head output

Able to pay attention on different things

Useful for NLP:

Different aspects of language can be found in different parts of a sentence

Multi-head Attention

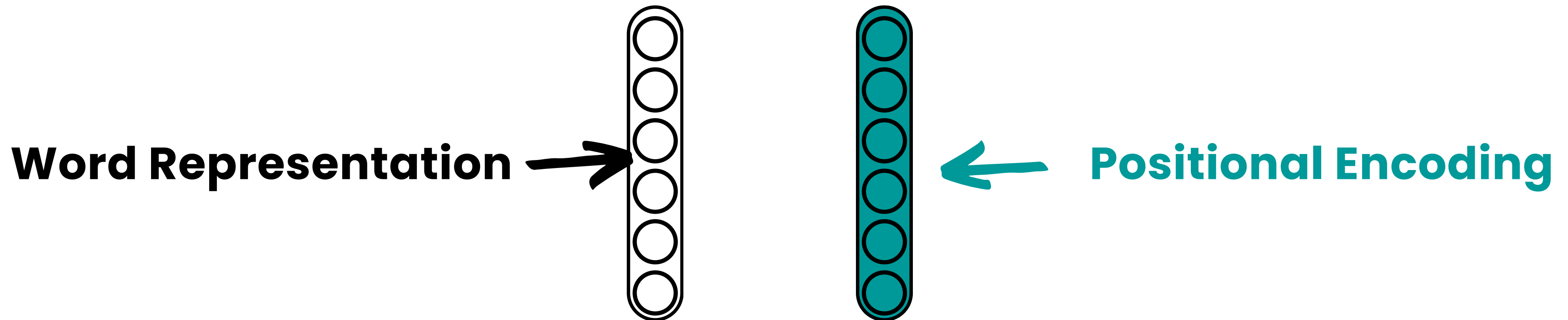


Positional Encoding

Positional Encoding: Desired Properties

Same dimension as word representations

→ They can be summed



Positional Encoding: Desired Properties

Same dimension as word representations

→ They can be summed

Provide a unique encoding for each position in a sentence

Consistent Distance

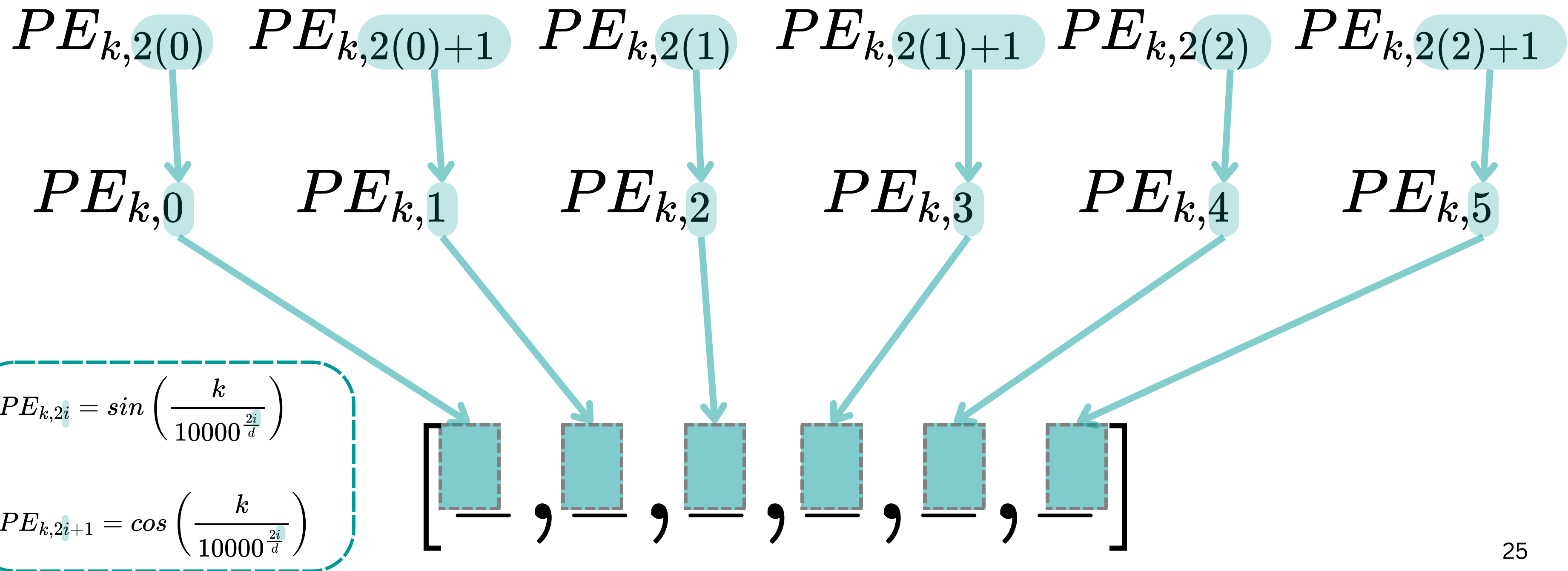
→ Between any two positions in a sentence.

→ For all sentence lengths

Able to handle longer sentences without effort

→ Each word = a number in an infinite interval → **problematic**

Trigonometric positional encodings



Example

“ Where are you going? ”

Let's calculate a 6 dimensional positional encoding of “you”

We are at the third dimensions (index 2) so $k = 2$

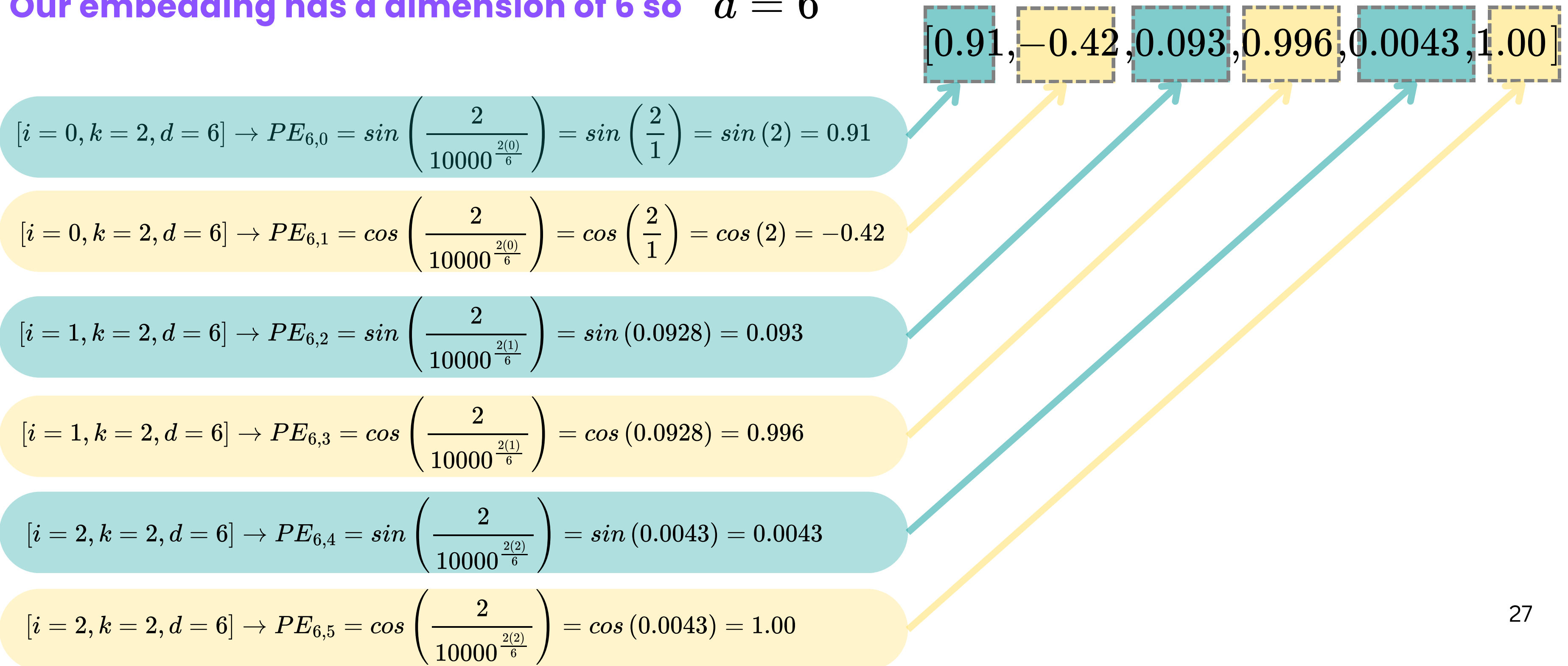
Our embedding has a dimension of 6 so $d = 6$

Example

“ Where are you going? ”

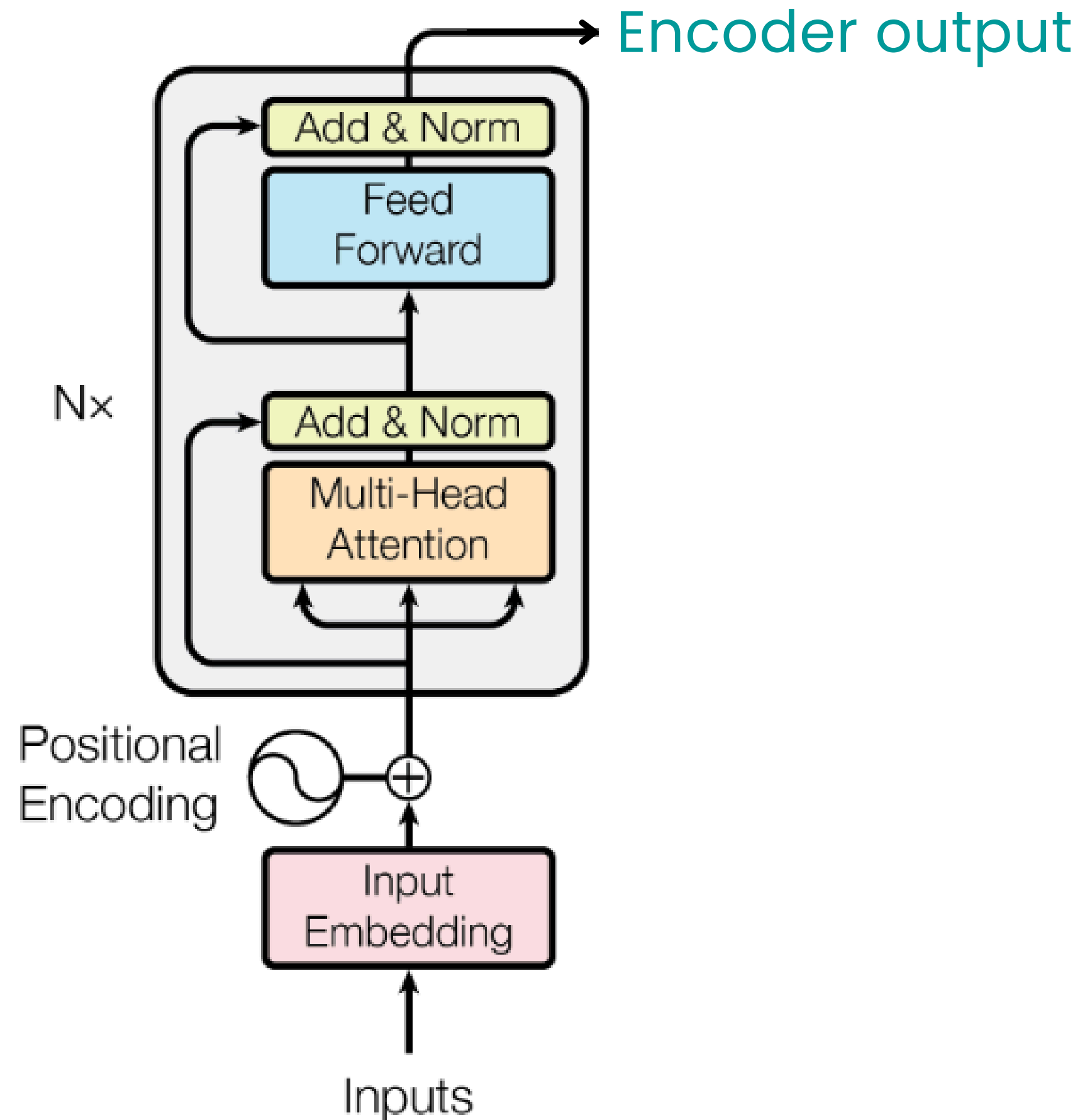
We are at the third dimensions (index 2) so $k = 2$

Our embedding has a dimension of 6 so $d = 6$

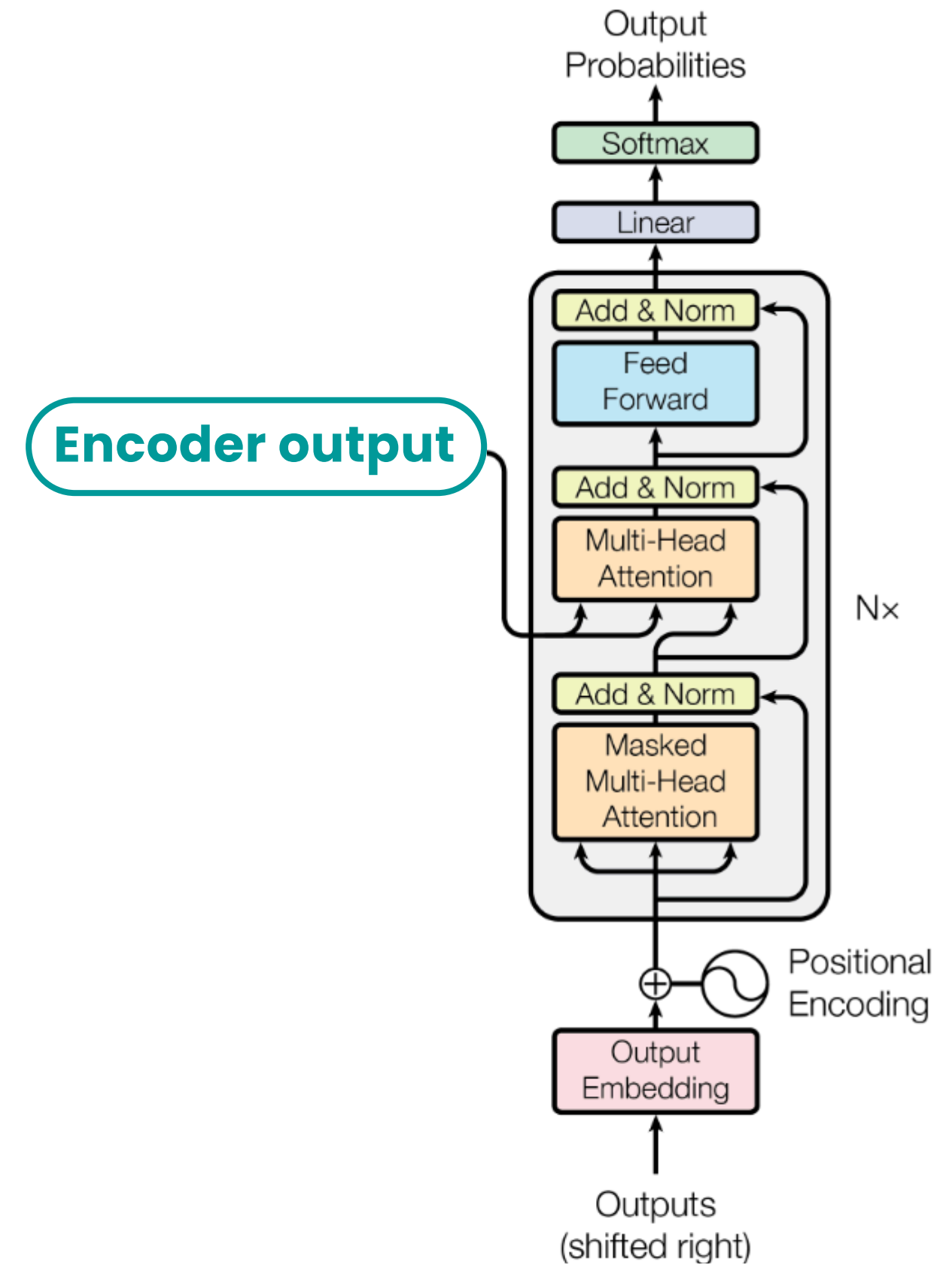


Full Transformer Architecture

Transformer: Encoder



Transformer: Decoder Components



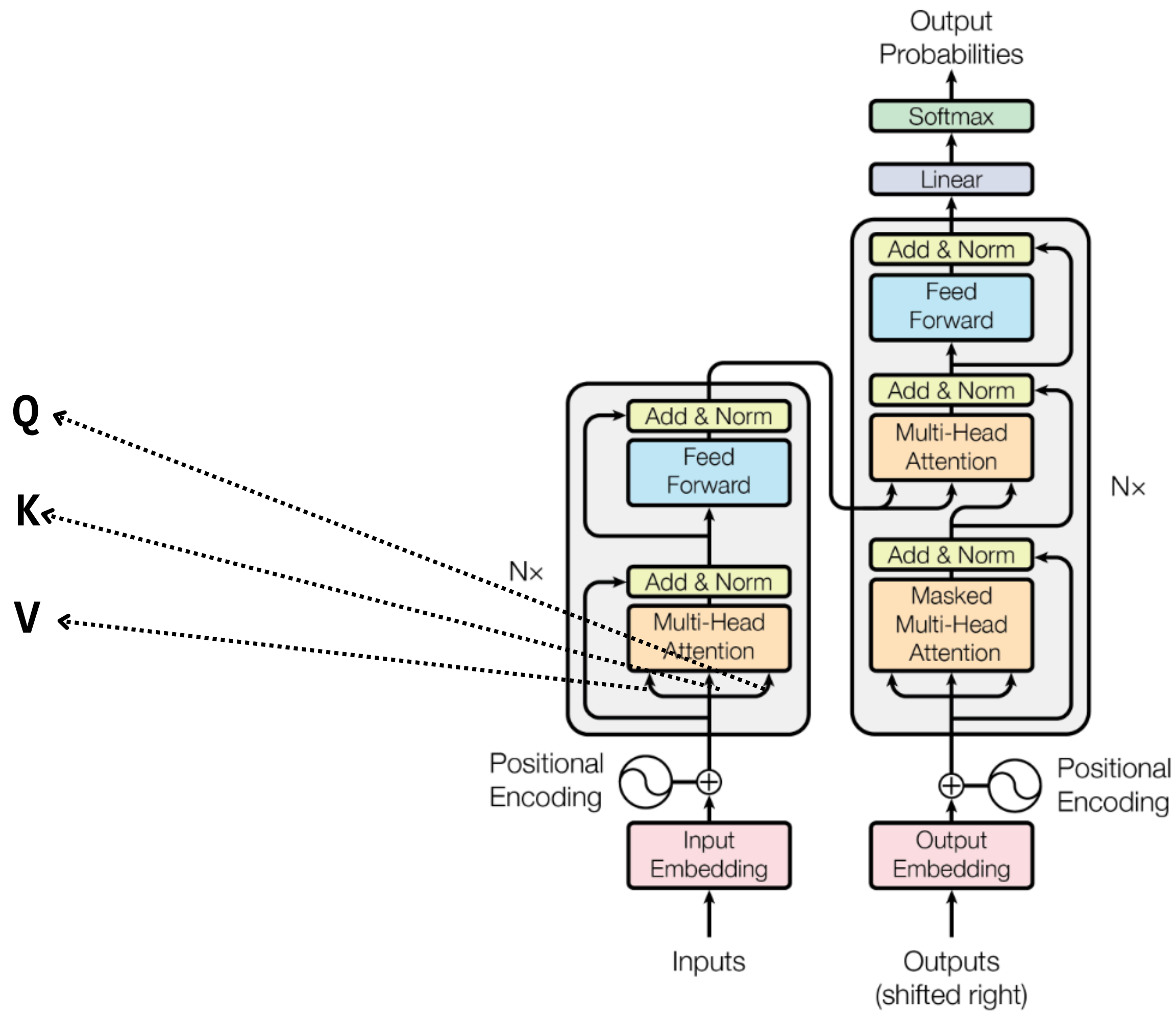


Figure 1: The Transformer - model architecture.

NMT vs LLM

Context

- **NMT able to deal with the entire sentence**

⇒ more fluent translations

- **LLM able to deal beyond the sentence level (paragraphs)**

⇒ even better fluency and better ability to deal with context-related phenomena

Language Model

- **NMT: guided by a language model (decoder: a language model with the source language information)**
- **LLM: language model itself**

Language model – history

- **NMT: (almost) unlimited**
⇒ more fluent translations
- **LLM: (almost) unlimited, much longer than for NMT**
⇒ better fluency, better handling context-level phenomena
+ possible to specify domain, register, etc. in the prompt

Words

- **NMT: (almost) unlimited**
⇒ more fluent translations
- **LLM: (almost) unlimited, much longer than for NMT**
⇒ better fluency, better handling context-level phenomena
+ possible to specify domain, register, etc. in the prompt

Source coverage constraint

- **NMT: no such constraint**

- ⇒ More omissions and additions

- ⇒ Repetitions

- ⇒ Hallucinations

- (content not related to any part of the source)

- **LLM: no connection to the source text at all!**

- ⇒ The problems even stronger (especially hallucinations)

Word alignments

- **NMT: no word alignments**
⇒ Attention provides something similar, but not really word alignments
- **LLM: no connection to the source text at all!**
⇒ No alignments at all (not even sentence-level ones)

Interpretation of errors

- **NMT: very difficult**
- **LLM: even more difficult**

Resources

Computational Requirements:

- NMT: CPUs impossible or too slow $\Rightarrow$ GPUs necessary
- LLM: vast amount of GPUs necessary

Data Requirements:

- NMT requires more training data
- NMT is more sensitive to noisy data (SMT deals with the noise by assigning low probabilities)
- LLM requires vast amounts of (monolingual) data

NMT vs LLM: Overview of differences

NMT	LLM
sentence-level context	beyond sentence-level
Guided by LM (long history)	LM itself (very long history) possible to specify many aspects in the “prompt”
sub-word units	sub-word units
fluency: better handles both syntax and morphology	fluency: even better, more natural
no coverage constraints, over/under generation	not bound to the source text at all
no word alignments	no alignments at all
“black box”, difficult to interpret errors	even more difficult to interpret errors
impossible (or too slow)on CPUs ⇒ GPUs	requires vast resourcesto work properly

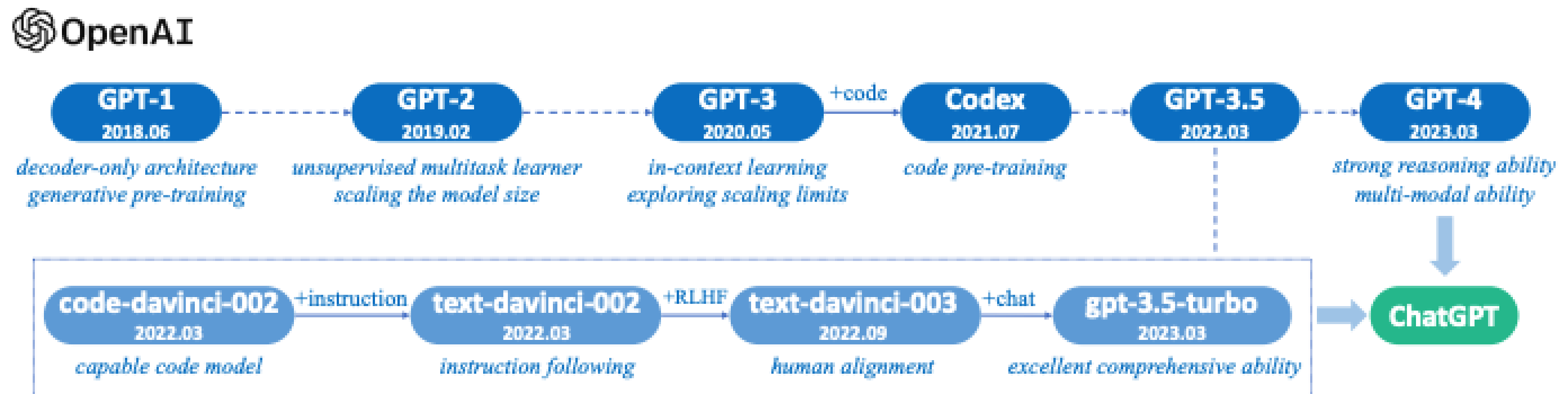
Generative Pre-trained Transformer

GPT

The GPT family of language models are language models trained using a transformer to predict the next word in a sequence, given the previous words

GPT: a brief history

- **GPT-1, 2018:** trained on Books Corpus, 117M parameters
- **GPT-2, 2019:** trained on outbound reddit links, 1.5B parameters
- **GPT-3, 2020:** trained on web text, books, Wikipedia, 175B parameters
- **GPT-3.5, 2022:** code included, human-in-the-loop reinforcement learning (so-called aligned language models)
- **GPT-4, 2023** and **GPT-4o, 2024:** multimodal
ChatGPT is a public interface to **GPT models**



Prompt Engineering

- From GPT-3 onwards, the emphasis has changed from fine-tuning a model on a task (as is done with BERT) to **prompting** a language model.
- Prompting a language model is essentially just asking the model to perform the task of interest by providing a suitable instruction or prompt
- For example, if the task is sentiment analysis of restaurant reviews, the prompt might be the text of the review followed by the question, *"What is the sentiment of this review?"*

"The Ivy is the most overrated restaurant I've ever been to. Avoid at all costs."

"What is the sentiment of this review?"

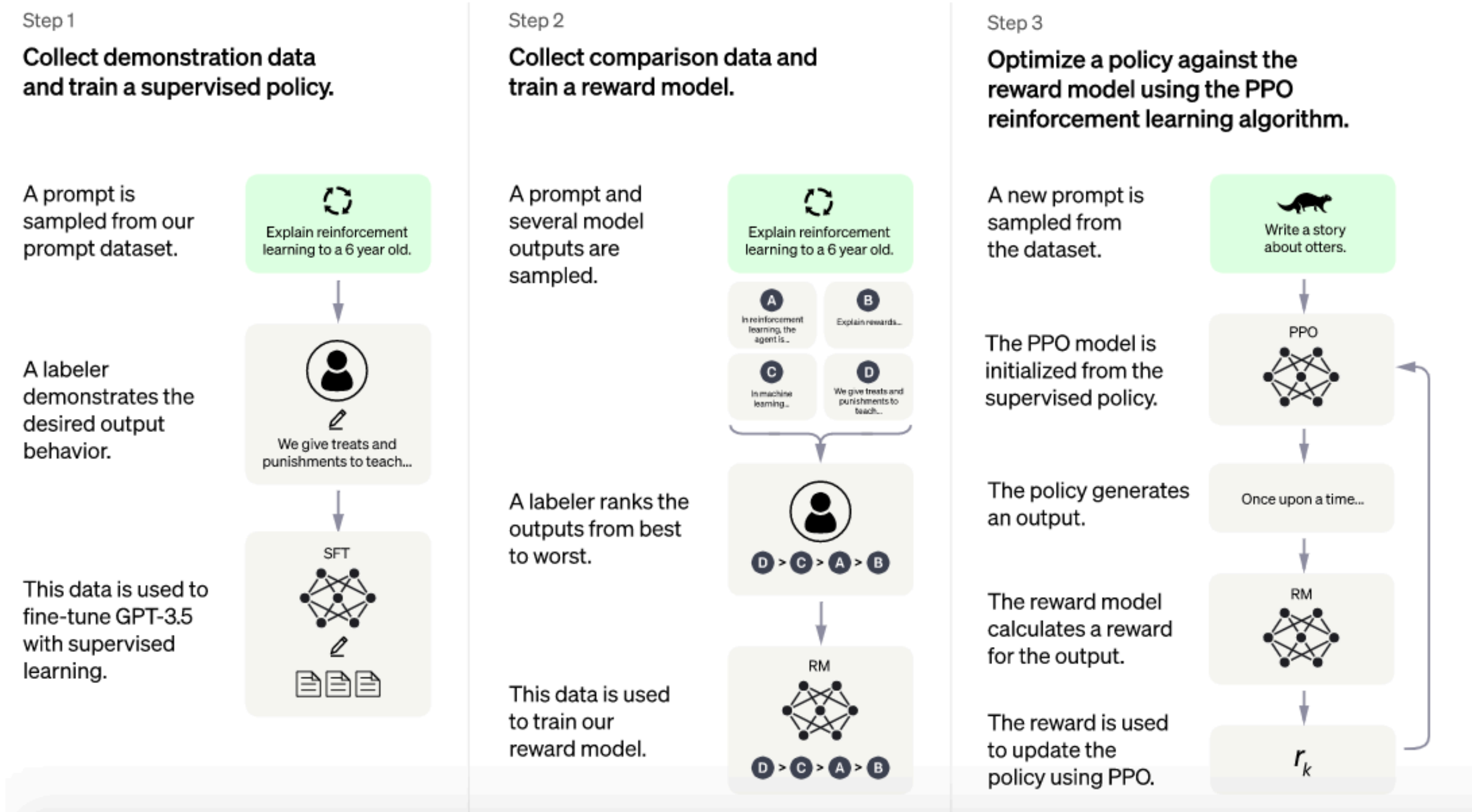
Few-shot versus zero-shot prompting

- **Zero-shot**: no examples are provided, e.g.
The Ivy is the most overrated restaurant I've ever been to. Avoid at all costs.
What is the sentiment of this review?
- **Few-shot** Include some labelled examples in the prompt, e.g.
 1. *Robertas is good on a weekday night but too busy on weekends. It's also overpriced.*
What is the sentiment of this review? Negative
 2. *Sophies has great food and spectacular views of Dublin.*
*What is the sentiment of this review? Positive**The Ivy is the most overrated restaurant I've ever been to. Avoid at all costs.*
What is the sentiment of this review?

Aligning a language model

- Take a large language model and make it better at following instructions and engaging in dialogue
- How?
- By introducing a training phase that uses the following data:
 - Instructions/prompts to the LLM with responses written, not by the LLM, but by human annotators (i.e. people!)
 - LLM responses scored by human annotators

Human-in-the-loop reinforcement learning



Large Language Models in 2023

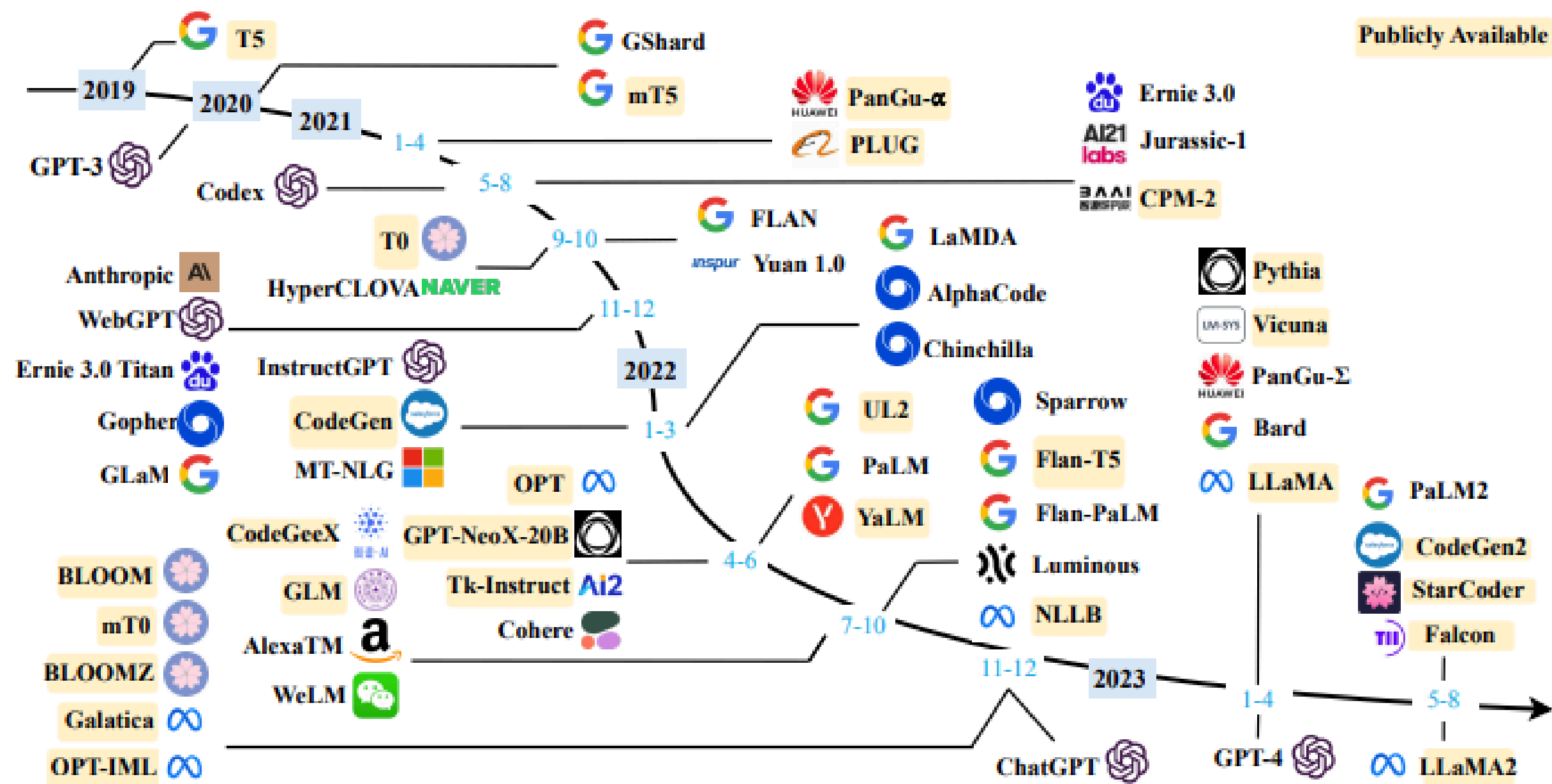


Fig. 2: A timeline of existing large language models (having a size larger than 10B) in recent years. The timeline was established mainly according to the release date (*e.g.*, the submission date to arXiv) of the technical paper for a model. If there was not a corresponding paper, we set the date of a model as the earliest time of its public release or announcement. We mark the LLMs with publicly available model checkpoints in yellow color. Due to the space limit of the figure, we only include the LLMs with publicly reported evaluation results.

Recent Innovations

- 1.Chain-of-thought prompting
- 2.Retrieval-oriented generation

Chain-of-thought prompting

A prompting strategy which encourages the model to break a problem into sub problems

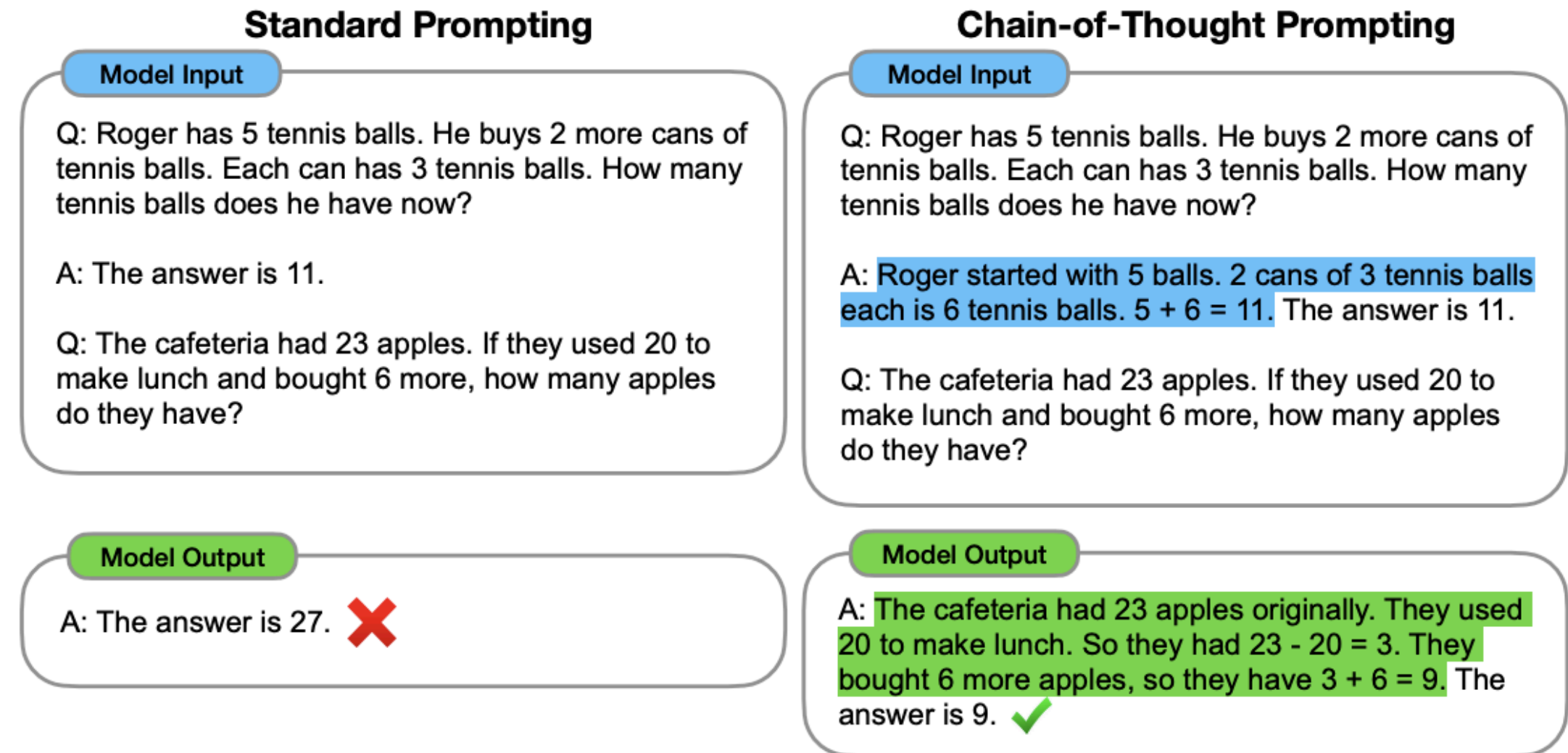


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

Retrieval-oriented Generation

- Incorporates an external knowledge source
- The input prompt is used to query the external knowledge source.
- The retrieved data is used by the LLM when generating a response
- Allows for more reliable, controlled generation

That's it for now!

DCU

Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University